
Datenbank-Entwurf

BITLC | Tom Selig
3. August 2026

Entity-Relationship-Modell (ERM)

Grundlagen

- Hilfsmittel zum Entwurf komplexer Datenbanken
- Entwickelt von Dr. Peter Chen (1976)
- Vermeidung/Beseitigung von Redundanzen

Entity-Relationship-Modell:

Beseitigung von Redundanzen über Grenzen von Relationen hinweg.

Normalisierung:

Beseitigung von Redundanzen innerhalb einer Relation.

- Abstrakte Darstellung von Beziehungen zwischen Objekten
- Noch keine Festlegung auf ein Datenbankmodell

Grundlegende Komponenten

Aus Konzepten, Beispielen und Diskussionen gehen hervor:

- Entitäten

Eindeutig unterscheidbare Objekte der realen Welt (real oder virtuell)
=> *der Autor Kai Günster, das Buch „Einführung in Java“*

- Eigenschaften

Eigenschaften, die im Kontext für eine Entität relevant sind
=> *Die Bankverbindung des Autors, der Preis des Buchs*

- Beziehungen

Verknüpfung oder Zusammenhang zwischen zwei oder mehr Entitäten
=> *Der Autor Kai Günster hat das Buch „Einführung in Java“ geschrieben*

Grundlegende Komponenten

Im Entity-Relationship-Modell werden daraus:

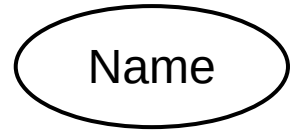
- Entitätstypen

Zusammenfassung gleichartiger Entitäten zu einem Typ
=> *ein Autor, ein Buch, eine Bank*



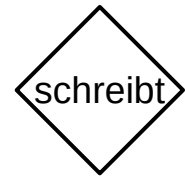
- Attribute

Typisierung gleichartiger Eigenschaften eines Entitätstypen
=> *Name und Konto-Nr. beim Entitätstyp Autor*

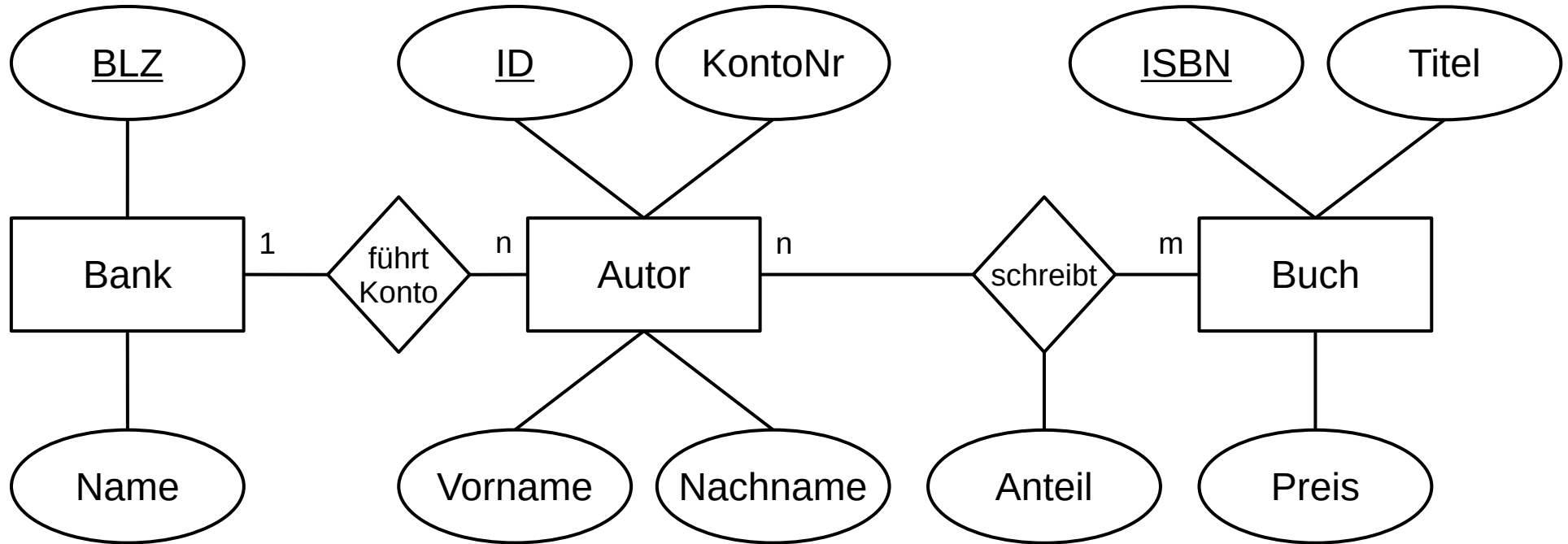


- Beziehungstypen

Typisierung gleichartiger Beziehungen zwischen Entitätstypen
=> *Autor schreibt Buch*



Beispiel



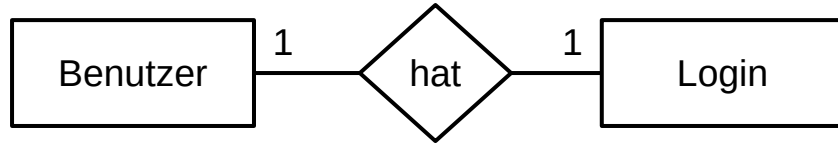
Kardinalitäten

- Mengenangaben, die für jeden Beziehungstyp festlegen, wie viele Entitäten des einen Entitätstyps mit genau einer Entität des anderen Entitätstyps in Beziehung stehen können oder müssen.
- Werden an die Verbindungslinien zum Beziehungstyp geschrieben.
- Zur Definition müssen immer **zwei** Sätze gebildet werden:
 - Eine** Entität des **ersten** Entitätstypen steht mit **X** Entitäten des **zweiten** Entitätstypen in Verbindung.
 - Eine** Entität des **zweiten** Entitätstypen steht mit **Y** Entitäten des **ersten** Entitätstypen in Verbindung.
- Verschiedene Formen der Notation sind möglich

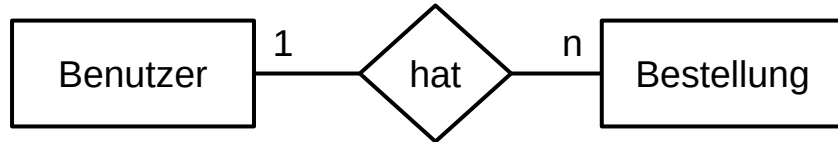
Chen-Notation

- Nach Dr. Peter Chen (1976) $\Rightarrow$ IHK-Standard
- Zeigt immer nur die maximal mögliche Anzahl an Verbindungen
- Mögliche Ausprägungen:
 - 1 $\Rightarrow$ 0 oder 1
 - n, m $\Rightarrow$ 0 bis ∞
- Mögliche Beziehungen:
 - 1:1 $\Rightarrow$ Eher selten, meist aus Gründen der Komplexität oder Sicherheit
 - 1:n $\Rightarrow$ Häufigste Form in relationalen Datenbanken
 - n:m $\Rightarrow$ In relationalen Datenbanken nicht direkt abbildbar

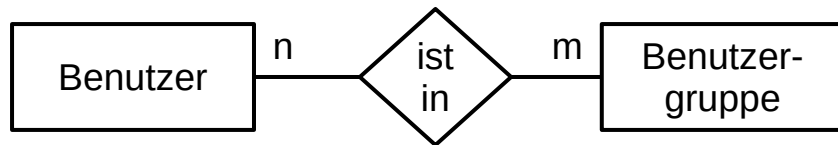
Chen-Notation



Ein Benutzer hat keinen oder einen Login. Ein Login gehört zu keinem oder einem Benutzer.



Ein Benutzer hat keine, eine oder viele Bestellungen. Eine Bestellung gehört zu keinem oder einem Benutzer.



Ein Benutzer ist in keiner, einer oder vielen Benutzergruppen. Eine Benutzergruppe enthält keinen, einen oder viele Benutzer.

Modifizierte Chen-Notation

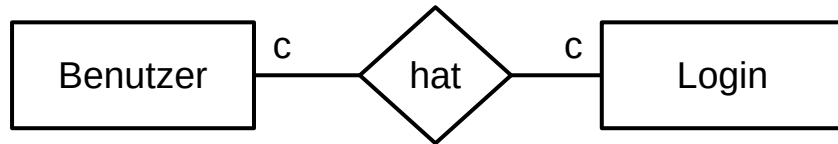
Erlaubt eine feinere Unterscheidung durch geänderte Symbol-Bedeutungen:

1: genau 1

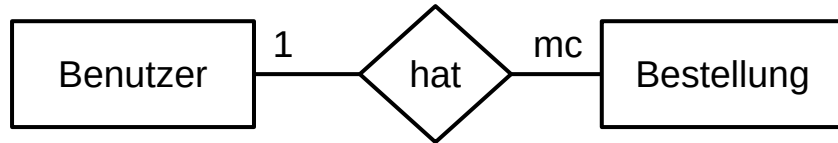
c: 0 oder 1

m: mindestens 1

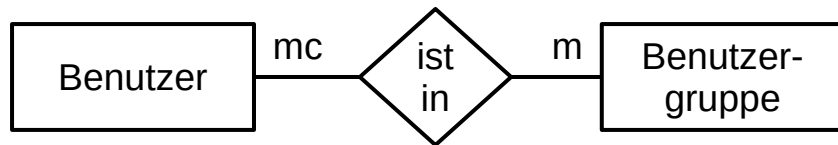
mc: beliebig viele



Ein Benutzer hat keinen oder einen Login. Ein Login gehört zu keinem oder einem Benutzer.



Ein Benutzer hat keine, eine oder viele Bestellungen. Eine Bestellung gehört zu genau einem Benutzer.



Ein Benutzer ist in einer oder vielen Benutzergruppen. Eine Benutzergruppe enthält keinen, einen oder viele Benutzer.

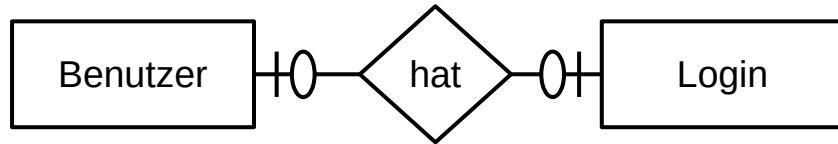
Martin-/Crow's-Foot-Notation

Erlaubt eine feinere Unterscheidung durch Angabe von Minimum und Maximum:

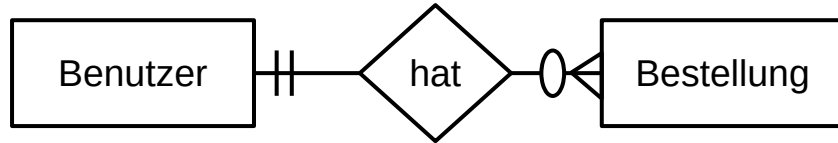
0: keine

|: eine

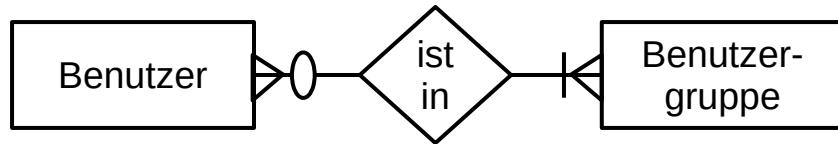
≥: viele



Ein Benutzer hat keinen oder einen Login. Ein Login gehört zu keinem oder einem Benutzer.



Ein Benutzer hat keine, eine oder viele Bestellungen. Eine Bestellung gehört zu genau einem Benutzer.



Ein Benutzer ist in einer oder vielen Benutzergruppen. Eine Benutzergruppe enthält keinen, einen oder viele Benutzer.

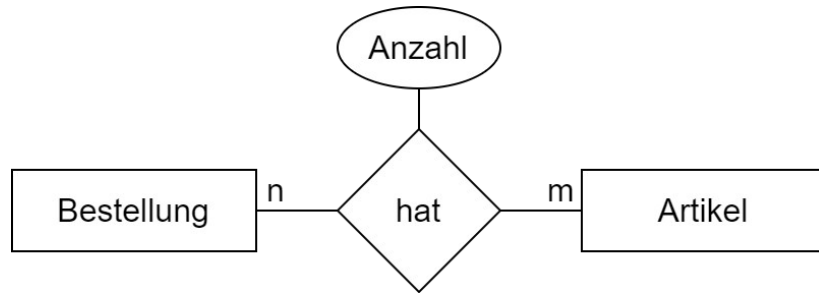
Besonderheiten

- Attribute können auch an den Beziehungstypen hängen.
 - Das kommt häufiger bei m:n-Beziehungen vor.
 - Beispiel:
- Anzahl ist von der Bestellung **und** vom Artikel abhängig:

Hängt man das Attribut an die Bestellung, gilt dies immer für diese Bestellung. Alle Artikel in der Bestellung hätten die gleiche Anzahl.

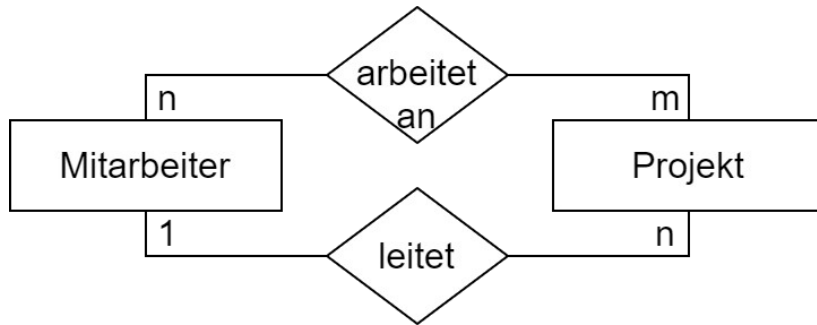
Hängt man das Attribut an den Artikel, gilt dies immer für diesen Artikel. Der Artikel hätten in allen Bestellungen die gleiche Anzahl.

- Also: Anzahl an die Beziehung



Besonderheiten

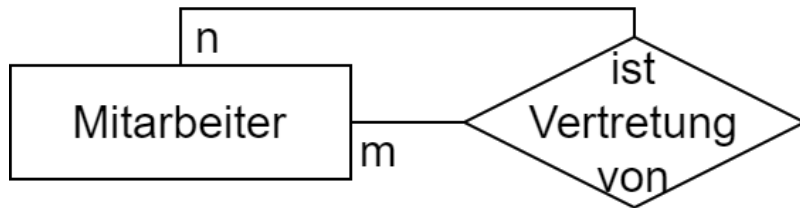
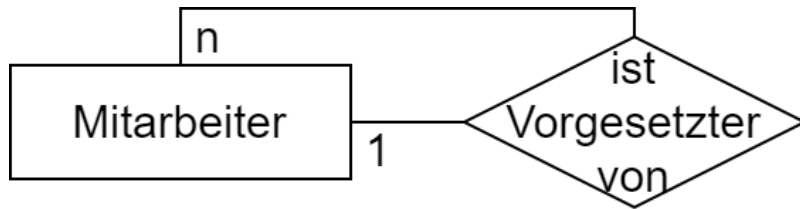
- Zwischen zwei Entitätstypen kann es auch mehr als eine Beziehung geben.
- Beispiel:



- Es gilt:
Ein Mitarbeiter kann an mehreren Projekten arbeiten, an einem Projekt können mehrere Mitarbeiter arbeiten.
- Es gilt auch:
Ein Mitarbeiter kann mehrere Projekte leiten, ein Projekt wird von maximal einem Mitarbeiter geleitet.
- Also: 2 Beziehungen (n:m, 1:n)

Besonderheiten

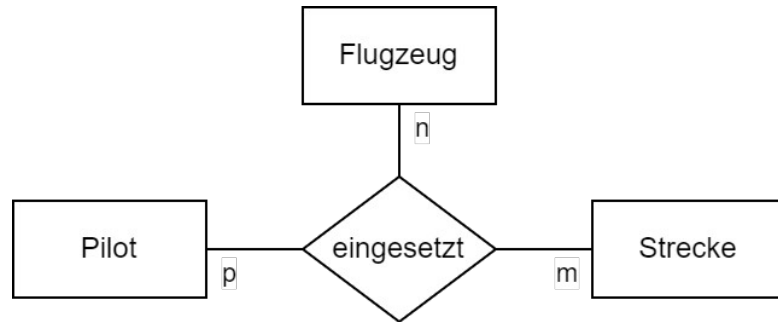
- Entitätstypen können auch rekursive Beziehungen haben.
- Dabei hat der Entitätstyp eine Beziehung zu sich selbst:



- Oberes Beispiel:
Ein Mitarbeiter kann Vorgesetzter von mehreren Mitarbeitern sein, ein Mitarbeiter hat maximal einen Vorgesetzten.
- Unteres Beispiel:
Ein Mitarbeiter kann mehrere andere Mitarbeiter vertreten, ein Mitarbeiter kann von mehreren anderen Mitarbeitern vertreten werden.

Besonderheiten

- Beziehungen können auch zwischen mehr als zwei Entitätstypen bestehen.
- Bei drei Entitätstypen spricht man von einer ternären Beziehung.



- Bestimmung der Kardinalitäten:

Ein Pilot kann mit **einem** Flugzeug auf **mehreren** Strecken eingesetzt werden.

Ein Pilot kann auf **einer** Strecke mit **mehreren** Flugzeugen unterwegs sein.

Ein Flugzeug kann auf **einer** Strecke von **mehreren** Piloten geflogen werden.

Tabellenmodell

Grundlagen

- Das Entity-Relationship-Modell ist noch nicht auf ein Datenbankmodell festgelegt.
- Das Tabellenmodell gehört zum relationalen Datenbankmodell.
- Besonderheiten des relationalen Modells werden berücksichtigt.
n:m-Beziehungen sind im relationalen Modell nicht möglich.
- Weitere Details zu den Attributen können festgelegt werden.
Datentypen und evtl. Einschränkungen für die Tabellen-Spalten.
- Beziehungen werden konkret definiert
Primär- und Fremdschlüssel werden vergeben.

Schlüssel

- Schlüsselkandidat

Eine minimale Menge von Attributen (Spalten) einer Relation (Tabelle), die jedes Tupel (Datensatz) eindeutig identifiziert.

Es kann mehrere Schlüsselkandidaten in einer Relation geben.

- Primärschlüssel (primary key, PK)

Einer der Schlüsselkandidaten, der für die Relation verwendet wird.

Alle anderen Schlüsselkandidaten werden Alternativschlüssel genannt.

Ist eine Spalte als Primärschlüssel deklariert, stellt die Datenbank die Eindeutigkeit sicher.

Schlüssel

- Fremdschlüssel (foreign key, FK)

Eine Spalte oder eine Kombination von Spalten, die auf den Primärschlüssel einer anderen (oder der gleichen) Relation verweist.

Wenn der Primärschlüssel aus mehreren Spalten besteht, muss auch ein Fremdschlüssel, der darauf verweist, aus der gleichen Kombination von Spalten bestehen.

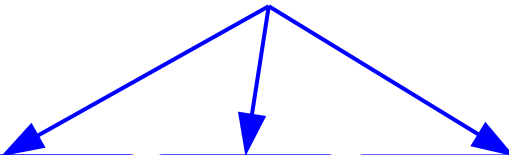
- Referentielle Integrität

In einer Fremdschlüssel-Spalte können nur Werte eingetragen werden, die auch im zugehörigen Primärschlüssel existieren.

Werte, die im Fremdschlüssel verwendet werden, können im zugehörigen Primärschlüssel in der Regel nicht gelöscht werden.

Schlüssel

Schlüsselkandidaten

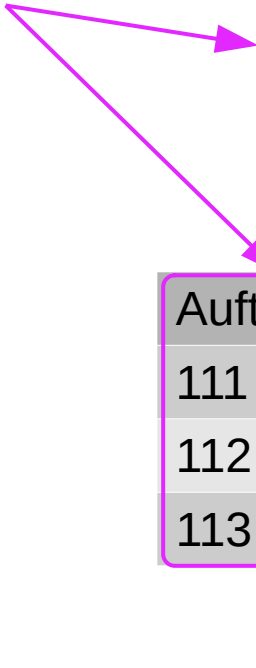


Name	Symbol	Ordnungszahl	Jahr
Wasserstoff	H	1	1766
Helium	He	2	1895
Lithium	Li	3	1817
Beryllium	Be	4	1797

Primärschlüssel



KundeID	Vorname	Nachname
123	Gerda	Guth
234	Max	Muster
345	Ute	Hansen



AuftragID	KundeID	Beschreibung
111	123	...
112	123	...
113	345	...

Fremdschlüssel

Schlüssel

- natürliche/sprechende Schlüssel

Schlüssel, deren Attribute auf natürliche Weise bereits in der Relation vorkommen. Sie haben auch in der realen Welt Bedeutung.

Sind in der Regel schwierig, da Eindeutigkeit selten garantiert werden kann.

Z.B.: Fahrgestellnummer, Sozialversicherungsnummer

- künstliche Schlüssel/Surrogatschlüssel

Ein künstlich erzeugtes, in der Relation zuvor nicht vorkommendes Attribut, das die Datensätze der Relation eindeutig identifiziert.

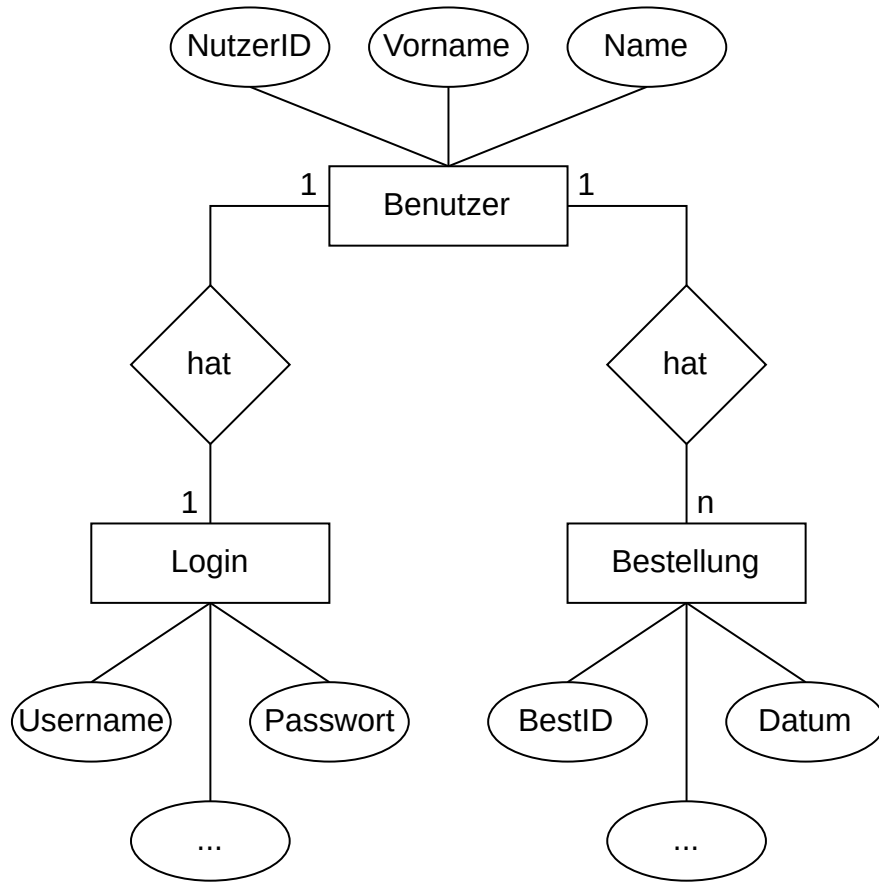
Wird oft verwendet, um Schlüssel mit sehr vielen Attributen zu ersetzen.

Z.B.: Fortlaufende Nummer, GUID

Vom ERM zum Tabellenmodell

- Jeder Entitätstyp wird zu einer Tabelle.
 - Attribute werden zu Spalten dieser Tabelle.
 - Datentypen der Spalten und ggf. Einschränkungen bestimmen.
 - Wenn noch nicht vorhanden, muss ein Primärschlüssel ergänzt werden.
- 1:1- und 1:n-Beziehungen werden durch Fremdschlüssel aufgelöst.
 - 1:1 Fremdschlüssel in genau einer der beiden Tabellen anlegen. Der Fremdschlüssel ist dort dann gleichzeitig der Primärschlüssel und verweist auf den Primärschlüssel der anderen Tabelle.
 - 1:n Fremdschlüssel immer in der n-Tabelle anlegen. Er verweist auf den Primärschlüssel der 1-Tabelle.

Vom ERM zum Tabellenmodell



Benutzer				
Name	Typ	Länge	PK	FK
NutzerID	Integer	10	X	
Vorname	Char	50		
Nachname	Char	50		

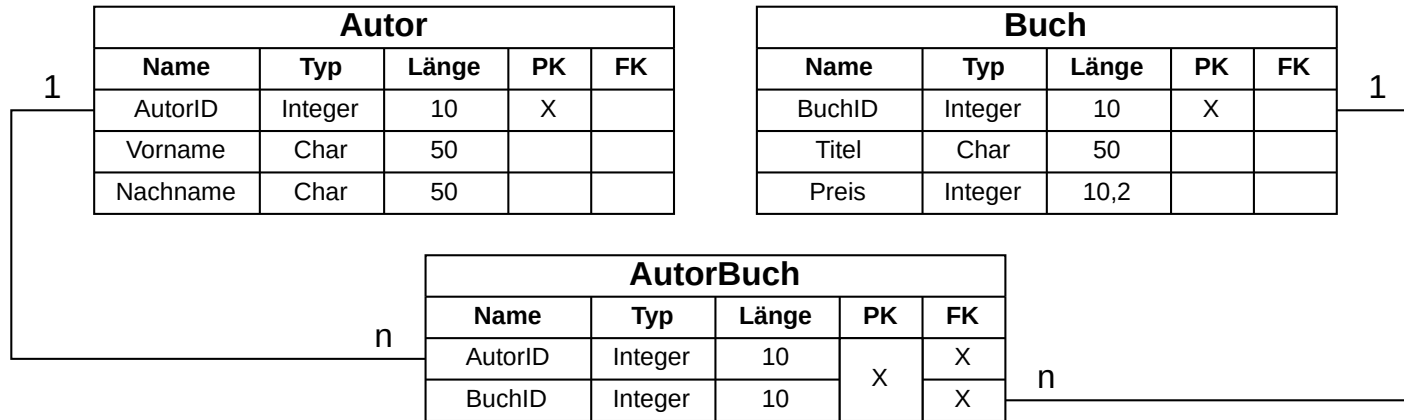
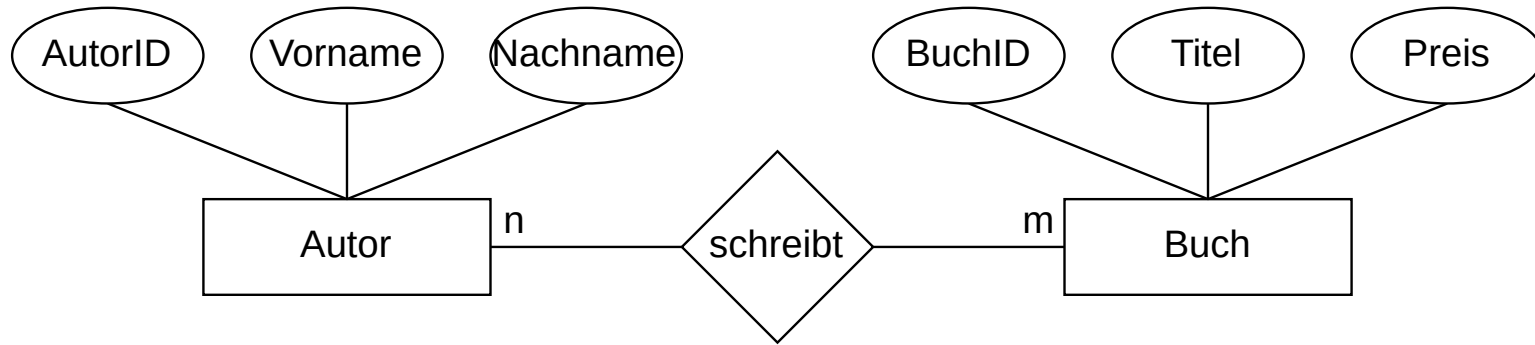
Login				
Name	Typ	Länge	PK	FK
NutzerID	Integer	10	X	X
Username	Char	20		
Passwort	Char	20		
...	...	...		

Bestellung				
Name	Typ	Länge	PK	FK
BestID	Integer	10	X	
NutzerID	Integer	10		X
Datum	Date	8		
...	...	...		

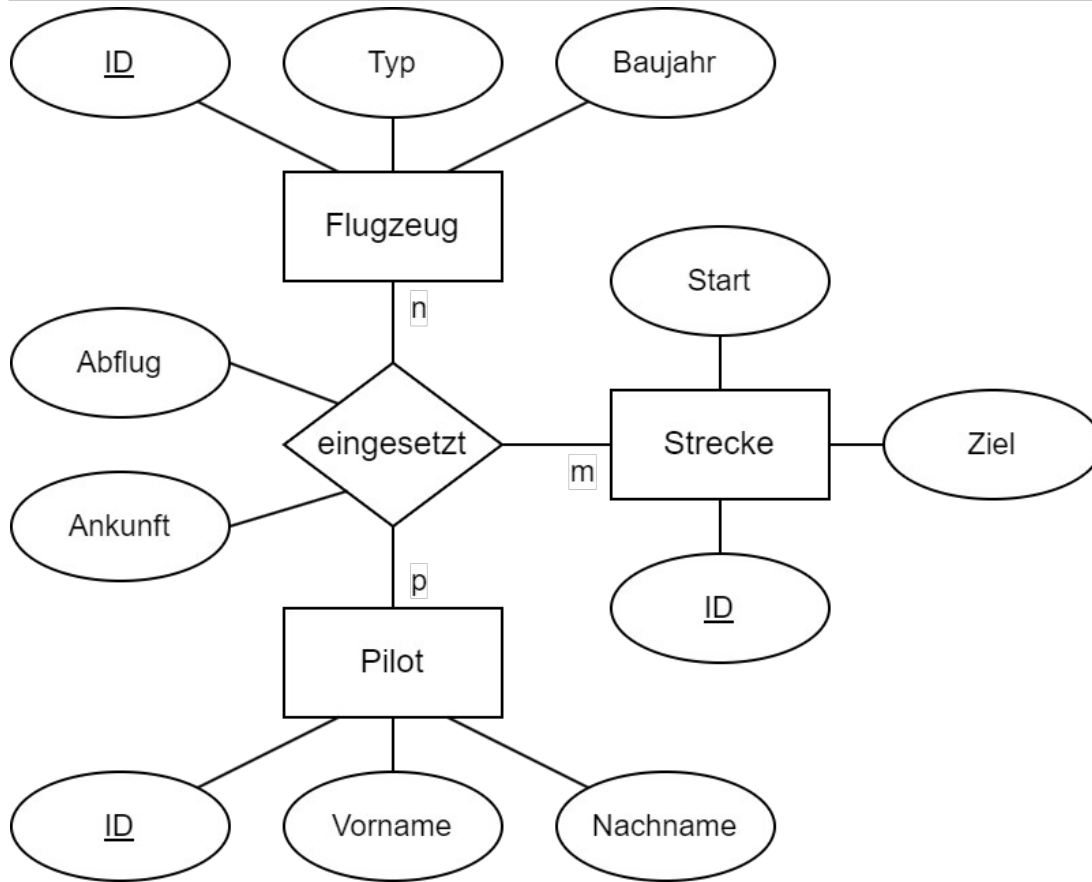
Vom ERM zum Tabellenmodell

- m:n-Beziehungen werden über eine Zwischentabelle aufgelöst.
Die Primärschlüssel der beiden beteiligten Tabellen werden in eine neue Zwischen- oder Assoziations-Tabelle übertragen.
Sie bilden gemeinsam den Primärschlüssel der neuen Tabelle.
Jeder für sich ist gleichzeitig ein Fremdschlüssel in die jeweilige Originaltabelle.
Dadurch entstehen zwei 1:n-Beziehungen, das n steht dabei immer an der neuen Zwischentabelle.
- Beziehungen zwischen den Tabellen werden eingezeichnet.
Linie zwischen Fremdschlüssel und Primärschlüssel, mit Kardinalitäten.

Vom ERM zum Tabellenmodell



Vom ERM zum Tabellenmodell



Flugzeug		
Name	PK	FK
ID	X	
Typ		
Baujahr		

Strecke		
Name	PK	FK
ID	X	
Start		
Ziel		

Pilot		
Name	PK	FK
ID	X	
Vorname		
Nachname		

Einsatz		
Name	PK	FK
Flugzeug_ID	X	X
Strecke_ID		X
Pilot_ID		X
Abflug		
Ankunft		

**Achtung:
Primärschlüssel!**

Vom ERM zum Tabellenmodell

- Attribute an 1:1-Beziehungen

Kommen in eine der beiden Tabellen, je nach dem wo es besser „passt“.

- Attribute an 1:n-Beziehungen

Kommen immer in die n-Tabelle.

Nur so können unterschiedliche Werte für die unterschiedlichen Beziehungen realisiert werden.

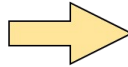
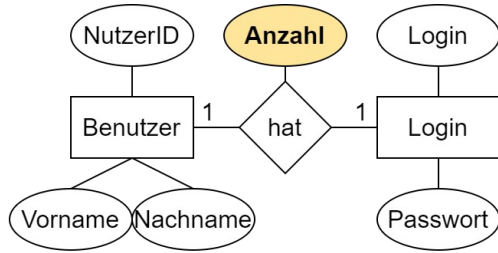
- Attribute an n:m-Beziehungen

Kommen immer in die neu entstandene Zwischentabelle.

Da sie immer von beiden Entitäten abhängen, können sie nicht in einer der Originaltabellen untergebracht werden.

Vom ERM zum Tabellenmodell

1:1



Benutzer			
Name	PK	FK	
NutzerID	X		
Vorname			
Nachname			

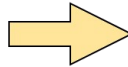
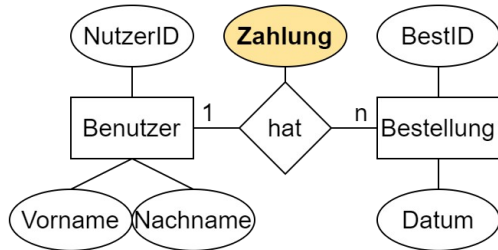
Login			
Name	PK	FK	
NutzerID	X	X	
Username			
Passwort			
Anzahl			



Benutzer			
Name	PK	FK	
NutzerID	X		
Vorname			
Nachname			
Anzahl			

Login			
Name	PK	FK	
NutzerID	X	X	
Username			
Passwort			

1:n



Benutzer			
Name	PK	FK	
NutzerID	X		
Vorname			
Nachname			

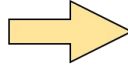
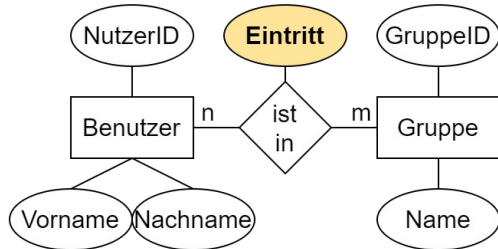
Bestellung			
Name	PK	FK	
BestID	X		
NutzerID		X	
Datum			
Zahlung			



Benutzer			
Name	PK	FK	
NutzerID	X		
Vorname			
Nachname			

Bestellung			
Name	PK	FK	
BestID	X		
NutzerID		X	
Datum			
Zahlung			

n:m



Benutzer			
Name	PK	FK	
NutzerID	X		
Vorname			
Nachname			

BenutzGruppe			
Name	PK	FK	
NutzerID	X	X	
GruppelD	X	X	
Eintritt			



BenutzGruppe			
Name	PK	FK	
NutzerID	X	X	
GruppelD	X	X	
Eintritt			

Gruppe			
Name	PK	FK	
GruppelD	X		
Name			

Normalisierung

Grundlagen

- Normalisierung ist ein Hilfsmittel beim Datenbankentwurf
 - Definition von insgesamt sechs aufeinander aufbauenden Normalformen
 - Definition von Regeln zur Erreichung der einzelnen Normalformen
- In der Praxis (IHK) sind die Normalformen 1 bis 3 relevant
- Normalisierung ist sinnvoll, aber nicht zwingend erforderlich
- Ziele der Normalisierung:
 - Vermeidung von Redundanzen, dadurch Vermeidung von Anomalien

Redundanzen und Anomalien

- Redundanz:

In Datenbanken spricht man von Redundanz, wenn Informationen oder Daten mehrfach vorhanden sind.

- Bsp.: Ein Kunde taucht mit Name und Adresse an mehreren Stellen der Datenbank auf.

- Anomalie:

Durch Redundanzen können Anomalien entstehen. Das bedeutet, dass mehrfach vorhandene Informationen sich widersprechen oder bestimmte Vorgänge nicht ausgeführt werden können oder bestimmte Vorgänge ungewollte Auswirkungen haben.

- Man unterscheidet Einfüge-, Änderungs- und Lösch-Anomalie.

Redundanzen und Anomalien

<u>PersonalNr</u>	Vorname	Nachname	...	<u>ProjektNr</u>	Bezeichnung	Tätigkeit	...
0001	Maria	Musterfrau	...	3	Reise zum Mars	Leiterin	...
0002	Max	Mustermann	...	1	Weltfrieden	Berater	...
0002	Max	Mustermann	...	2	Fusionsenergie	Leiter	...
0003	Kai	aus der Kiste	...	4	Weltherrschaft	Leiter	...
0004	Ingrid	Irgendwas	...	3	Reise zum Mars	Sachbearbeiterin	...
0004	Ingrid	Irgendwas	...	1	Weltfrieden	Leiterin	...

- Einfüge-Anomalie:

Neuer Mitarbeiter ohne Projekt oder ein neues Projekt ohne zugehörigen Mitarbeiter kann nicht eingefügt werden.

Es müssen immer beide Informationen eingefügt werden, da sonst der Primärschlüssel fehlerhaft wäre.

Redundanzen und Anomalien

<u>PersonalNr</u>	Vorname	Nachname	...	<u>ProjektNr</u>	Bezeichnung	Tätigkeit	...
0001	Maria	Musterfrau	...	3	Reise zum Mars	Leiterin	...
0002	Max	Mustermann	...	1	Weltfrieden	Berater	...
0002	Max	Mustermann	...	2	Fusionsenergie	Leiter	...
0003	Kai	aus der Kiste	...	4	Weltherrschaft	Leiter	...
0004	Ingrid	Irgendwas	...	3	Reise zum Mars	Sachbearbeiterin	...
0004	Ingrid	Irgendwas	...	1	Weltfrieden	Leiterin	...

- **Änderungs-Anomalie:**

Ändert sich der Nachname von Max Mustermann in Musterfrau, müssen alle Vorkommen geändert werden.

Sonst gäbe es zur PersonalNr 0002 zwei verschiedene Nachnamen, die Daten wären inkonsistent.

Redundanzen und Anomalien

<u>PersonalNr</u>	Vorname	Nachname	...	<u>ProjektNr</u>	Bezeichnung	Tätigkeit	...
0001	Maria	Musterfrau	...	3	Reise zum Mars	Leiterin	...
0002	Max	Mustermann	...	1	Weltfrieden	Berater	...
0002	Max	Mustermann	...	2	Fusionsenergie	Leiter	...
0003	Kai	aus der Kiste	...	4	Weltherrschaft	Leiter	...
0004	Ingrid	Irgendwas	...	3	Reise zum Mars	Sachbearbeiterin	...
0004	Ingrid	Irgendwas	...	1	Weltfrieden	Leiterin	...

- **Lösch-Anomalie:**

Es können immer nur ganze Datensätze gelöscht werden, es kann nicht nur ein Mitarbeiter oder nur ein Projekt gelöscht werden.

Wird der Mitarbeiter 0003 gelöscht, gehen auch die Informationen zum Projekt 4 verloren.

Beispiel

Ein Verlag benötigt eine Datenbank um seine Autoren, deren Bankverbindung, sowie die von ihnen geschriebenen Bücher zu speichern.

Ein erster simpler Versuch mit einer Tabelle:

ID	Name	KontoNr	BLZ	Bank	ISBN1	Titel1	Preis1	Anteil1	ISBN2	Titel2	Preis2	Anteil2
----	------	---------	-----	------	-------	--------	--------	---------	-------	--------	--------	---------

ID	Eindeutige ID des Autors, <u>Primärschlüssel</u>
Name	Vor- und Nachname des Autors
KontoNr	Kontonummer des Autors
BLZ	Bankleitzahl der Bank des Autors
Bank	Name der Bank des Autors

ISBN1/2	ISBN-Nummer des Buchs
Titel1/2	Titel des Buchs
Preis1/2	Preis des Buchs
Anteil1/2	Beteiligung des Autors in Prozent

Probleme im Entwurf

- Leere Felder

Autoren mit nur einem Buch haben Leerfelder im Datensatz

- Wie sucht man ein Buch?

Das Buch „SQL Server 2016“ taucht in zwei Spalten auf

- Wie sucht man nach dem Nachnamen des Autors?

Suche muss einen Teilstring in der Spalte Name finden

- Was passiert bei einem Autor mit drei Büchern?

Datenbankschema muss erweitert werden

- *Daher: Eine Normalisierung wäre empfehlenswert*

1. Normalform (1NF)

- *Eine Relation ist in der ersten Normalform, wenn jedes Attribut nur atomare Werte enthält und es keine Wiederholungsgruppen gibt.*
- atomare Werte:
Der Wert ist nicht weiter (sinnvoll) aufteilbar
- Wiederholungsgruppen:
Spalten, die eine Liste mit Werten enthalten (z.B. mit Komma getrennt)
Mehrere Spalten mit gleichem Inhalt (Telefon1, Telefon2)
- Häufiger, sinnvoller Zusatz:
... und es einen Primärschlüssel gibt.

Widerspruch zur 1NF

- Das Attribut *Name* ist nicht atomar.
 - => Aufteilen in Attribute *Vorname* und *Nachname*
- Die Attribute *ISBN1/2*, *Titel1/2*, *Preis1/2* und *Anteil1/2* sind Wiederholungsgruppen.
 - => Autoren mit zwei Büchern auf zwei Datensätze verteilen
 - => Reduzierung auf Attribute *ISBN*, *Titel*, *Preis* und *Anteil*
- Der Primärschlüssel ist anschließend nicht mehr eindeutig.
 - => Primärschlüssel um Attribut *ISBN* erweitern

2. Normalform (2NF)

- *Eine Relation ist in der zweiten Normalform, wenn sie in der ersten Normalform ist und jedes Nichtschlüsselattribut voll funktional von allen Schlüsselkandidaten abhängig ist.*
- Nichtschlüsselattribut:

Alle Attribute, die nicht Teil eines Schlüsselkandidaten sind.
- Funktionale Abhängigkeit:

Ein Attribut X ist von einem Attribut Y funktional abhängig, wenn es zu jedem Wert von Y nur genau einen Wert von X geben kann.

Beispiel: Zu jedem Wert von ISBN gibt es nur genau einen Wert von Titel. Damit ist Titel funktional abhängig von ISBN.

2. Normalform (2NF)

- Voll funktionale Abhängigkeit:

Die funktionale Abhängigkeit kann auch zu einer Gruppe von Attributen bestehen. Voll funktionale Abhängigkeit bedeutet dann, das Attribut ist von der gesamten Gruppe abhängig, nicht nur von einer Teilmenge.

Beispiel 1: Der Primärschlüssel besteht aus ID und ISBN. Zu jedem Wert von ID gibt es nur einen Wert von Vorname, völlig egal was in der ISBN steht. Das Attribut Vorname ist also **nicht voll funktional abhängig** vom Primärschlüssel, sondern nur vom Attribut ID.

Beispiel 2: Zu jedem Wert von ID kann es unterschiedliche Werte von Anteil geben und zu jedem Wert von ISBN kann es ebenfalls unterschiedliche Werte von Anteil geben. Die Abhängigkeit besteht also nicht zu einem einzelnen Attribut, sondern zur Gruppe aus ID und ISBN. Das bedeutet, das Attribut Anteil **ist voll funktional abhängig** vom Primärschlüssel.

Widerspruch zur 2NF

- Die Attribute *Vorname*, *Name*, *KontoNr*, *BLZ* und *Bank* sind nur vom Attribut *ID* abhängig.
 - => Auslagern in eine eigene Relation „Autor“
 - => Attribut *ID* als Primärschlüssel mitnehmen
 - => Doppelte Zeilen entfernen
- Die Attribute *Titel* und *Preis* sind nur vom Attribut *ISBN* abhängig.
 - => Auslagern in eine eigene Relation „Buch“
 - => Attribut *ISBN* als Primärschlüssel mitnehmen
 - => Doppelte Zeilen entfernen

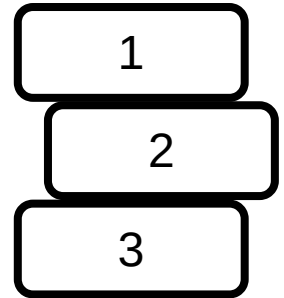
3. Normalform (3NF)

- *Eine Relation ist in der dritten Normalform, wenn sie in der zweiten Normalform ist und es keine funktionalen Abhängigkeiten zwischen Nichtschlüsselattributen gibt (keine transitiven Abhängigkeiten).*
- Transitive Abhängigkeit:

Attribut A ist von Attribut B funktional abhängig und
Attribut B ist wiederum von Attribut C funktional abhängig.
Dann ist Attribut A transitiv von Attribut C abhängig.

Beispiel:

Kiste 1 steht auf Kiste 2 und Kiste 2 steht auf Kiste 3.
Dann steht Kiste 1 auch auf Kiste 3, aber transitiv.



Widerspruch zur 3NF

- Das Attribut *Bank* in der Relation *Autor* ist funktional abhängig vom Attribut *BLZ*. Es ist also transitiv abhängig vom Primärschlüssel.
 - => Auslagern in eine eigene Relation *Bank*
 - => Attribut *BLZ* als Primärschlüssel mitnehmen
 - => Doppelte Zeilen entfernen